CS 330 – Fall 2026
Homework 7 – Programming Assignment 1
Dijkstra Tie-Breaking
Due Wednesday, October 21, 2026

Your Task Modify the provided implementation of Dijkstra's algorithm for directed graphs with non-negative edge weights.

For every vertex reachable from the source, your algorithm must find a path that:

1. Has the smallest possible total edge weight.
2. Among paths with that weight, uses the fewest edges.

For example, suppose $s \to a \to t$ has edge weights 1 and 1, and $s \to t$ has weight 2. Both paths have total weight 2, so your algorithm must choose $s \to t$, which uses only one edge.

Total weight takes priority over edge count: a path with smaller total weight must be preferred even if it uses more edges. If several paths have both the same minimum weight and the same minimum edge count, you may return any one of them.

Input and Output Keep the function signature:

```
def dijkstra(G, s):
```

Here, `G` is a `Graph` object from the supplied `simplegraphsPA1.py` library, and `s` is a node name stored as a string. You may assume that `s` belongs to `G`. Edge weights may be zero.

Return the pair `(distances, parents)`, where:

- `distances[v]` is the shortest-path distance from `s` to each reachable vertex `v`. In particular, `distances[s] = 0`.
- `parents[v]` is the vertex immediately preceding `v` on its chosen path from `s`. Set `parents[s] = None`.

The parent pointers must form a shortest-path tree rooted at `s`. Following parent pointers from any reachable vertex must lead back to `s`, and the resulting path must satisfy both requirements above.

You may omit unreachable vertices from both dictionaries. Alternatively, include them in both dictionaries with distance `math.inf` and parent `None`.

Starter Code and Efficiency Edit only `starter_code/dijkstra.py`. It contains a working implementation of ordinary Dijkstra's algorithm, but it does not enforce the fewest-edges rule.

The supplied graph library and priority queue are ready to use. Keep `simplegraphsPA1.py` and `MinIndexedPriorityQueue.py` unchanged; the autograder uses the provided versions.

Your modification should preserve Dijkstra's running time of $O((n + m) \log n)$, where n is the number of vertices and m is the number of edges.

Getting Started with Graphs in Python

The supplied `simplegraphsPA1.py` is a small, class-based graph library. Its main class, `Graph`, provides graph edits, CSV file input/output, and traversals such as BFS and DFS. All node names are stored as strings, and edge weights are stored as floats. Keep both supplied library files unchanged: the autograder uses the provided versions. No additional Python packages are needed for the local tests.

From the `starter_code/` directory, start `python3` (or a Jupyter notebook in that directory) and try:

```
>>> import simplegraphsPA1 as sg
>>> G = sg.Graph(directed=True)
>>> _ = G.add_edge("s", "a", 1.0)  # Adds the endpoints if needed.
>>> _ = G.add_edge("a", "t", 1.0)
>>> _ = G.add_edge("s", "t", 2.0)
>>> _ = G.add_node("isolated")
>>> G.n, G.m
(4, 3)
>>> G.adj["s"]
{'a': 1.0, 't': 2.0}
>>> "t" in G.adj["s"]  # Is there an edge s -> t?
True
>>> "s" in G.adj["t"]  # Directed edges need not go both ways.
False
>>> for v, weight in G.adj["s"].items():
...     print(v, weight)
...
a 1.0
t 2.0
```

The graph-editing methods return the graph itself; assigning that value to `_` above suppresses its display in the interpreter. In this example, the paths $s \to a \to t$ and $s \to t$ both have weight 2, so your modified algorithm must choose the one-edge path $s \to t$.

The adjacency structure `G.adj` is a dictionary of dictionaries: `G.adj[u][v]` is the weight of edge $u \to v$. Iterating over `G.adj` visits all nodes; iterating over `G.adj[u]` visits the outgoing neighbors of `u`. An isolated node has an empty adjacency dictionary. `G.n` counts nodes and `G.m` counts stored directed edges. For undirected graphs, each ordinary edge is stored in both directions and counted twice, but this assignment uses directed graphs.

You can also load a provided test graph:

```
>>> G = sg.Graph.read_csv("input_tests/test_input_1.csv", directed=True)
>>> with open("input_tests/test_input_1.source.txt") as f:
...     s = f.read().strip()
...
>>> from dijkstra import dijkstra
>>> distances, parents = dijkstra(G, s)
```

The graph CSV has a header `u,v,w`. A row with both `u` and `v` declares an edge from `u` to `v`; a blank weight means 1.0. A row with only `u` declares a node with no outgoing edges in that row. The companion `.source.txt` file contains the source node's name. All edge weights in this assignment are non-negative, including possibly zero.

Using the Priority Queue The priority queue provides these methods:

`insert(node, priority)` Insert a node with the given priority.

`decrease_key(node, priority)` Lower the priority of a node currently in the queue.

`extract_min()` Remove and return `(node, priority)` for a node of minimum priority.

`is_empty()` Return whether the queue is empty.

Priorities may be numbers or tuples. Use a consistent priority format throughout the queue.

Testing Your Solution From the `starter_code/` directory, run:

```
python3 run_dijkstra_sanity_tests.py
python3 run_dijkstra_file_tests.py
```

The sanity tests use small graphs that you can inspect by hand. The file-based tests read `input_tests/` and compare against `output_tests/`. They check returned distances and reconstruct paths from your parent pointers to check their total weights and edge counts. Your parent tree does not need to match the reference tree exactly.

The starter implementation is expected to pass some tests and fail others. Tests it already passes help you catch mistakes as you modify it. Passing the supplied tests does not guarantee correctness; you should create additional tests of your own.

Helpful Python Tip Python compares tuples from left to right. It compares the first entries, then uses the second entries only if the first entries are equal. For example:

```
(2, 5) < (3, 1)   # True: 2 < 3
(2, 3) < (2, 5)   # True: first entries tie, and 3 < 5
```

Submission Submit only `dijkstra.py` on Gradescope as PA1, the programming component of Homework 7, due Wednesday, October 21, 2026.